

RadicalRoots

Constructive conversion of algebraic numbers to radicals

A Python reference engine, a Wolfram Language front end,
and the mathematics behind their guarantees

Prepared for Vladimir Reshetnikov
8 September 2026 | Version 1.0

Abstract

An algebraic number has a radical expression precisely when the Galois group of its irreducible minimal polynomial is solvable. Turning this criterion into usable code requires considerably more than naming a Galois group: one must construct radical generators, represent every radicand in an earlier field, and select the correct complex branches. This report develops an explicit algorithm using splitting fields, prime cyclic quotients, rational nullspaces, and exact tower coordinates. A relative-automorphism construction avoids a second, potentially much larger, factorization after adjoining roots of unity. Finite trace-projector candidates substantially improve expression size.

The accompanying Python implementation contains this general construction, structural shortcuts, a non-executable JSON radical format, and a separate arithmetic-tower checker. The Wolfram Language package calls the engine and checks the final answer with `RootReduce`. The two motivating examples receive especially short solutions: a reciprocal sextic reduces to a cubic, and a quintic is a Dickson polynomial. The mathematical algorithm is complete with exact algebraic primitives and without resource limits. The software is a reference implementation, not a claim of unrestricted practical performance or a formally verified theorem prover.

Validation boundary. The Python suite was executed with SymPy 1.14.0: 46 of 46 tests passed. The Wolfram front end and its integration tests are supplied but were not executed: the available Wolfram service returned an HTTP 404 error, and no local Wolfram kernel was available. Exact details and a source fingerprint accompany the archive.

Contents

1	What problem is being solved?	2
1.1	The selected factor is essential	2
1.2	Four logically different outcomes	2
2	What the supplied notebook contributes	3
3	The Galois criterion and its constructive content	3
3.1	Choice of roots of unity	3
4	Concrete fields and their automorphisms	4
4.1	Constructing the splitting field	4
4.2	Rational power coordinates	4
4.3	A cheaper relative-automorphism construction	4
5	Prime cyclic quotients without subgroup enumeration	5
6	Constructing a radical generator by a nullspace	5
6.1	Smaller generators by finite trace projections	6
7	From field generators to a radical expression	6
7.1	Maintaining an exact tower basis	6
7.2	Choosing the complex branch	7
7.3	Separation-bound comparison	7
8	A separate arithmetic-tower checker	7
9	The complete algorithm and its guarantee	8
10	Finite shortcuts that matter in practice	9
10.1	Centered powers and polynomial composition	9
10.2	Dickson polynomials	9
10.3	Reciprocal lifts	9
10.4	Pair-sum resolvents	9
10.5	Cyclotomic polynomials and real traces	10
11	The two motivating examples	10
11.1	A sextic that is only a cubic plus a quadratic	10
11.2	The solvable quintic and correlated fifth roots	11
11.3	A genuine degree-five Galois construction	11
12	Using the software	12
12.1	Python interface	12
12.2	Wolfram Language interface	12
13	Validation and honest limitations	13
14	Directions for a stronger implementation	14

1 What problem is being solved?

The motivating question [1] asks why `ToRadicals` can leave solvable `Root` objects unchanged and whether a systematic workaround exists. Its documentation guarantees conversion for roots of polynomials of degree at most four, but explicitly notes that some possible radical representations are not found [2]. Failure of a particular conversion routine is therefore not a proof of impossibility.

The present input is a *specified exact algebraic number*. The Python interface receives a univariate polynomial in $\mathbb{Q}[x]$ and a selected root; the Wolfram interface first computes the rational minimal polynomial of the supplied exact value. Parameters, approximate coefficients, and roots of nonpolynomial analytic equations are outside the contract.

Definition 1.1. A radical expression over $\mathbb{Q}$ uses rational constants, arithmetic operations, and finitely many rational powers. We allow i , equivalently $\sqrt{-1}$. Every noninteger power has its principal complex meaning unless an explicit root-of-unity multiplier specifies another branch.

Neither `Root/CRootOf`, an unevaluated inverse function, nor a trigonometric constant is an acceptable final radical. In particular, $2\cos(2\pi/11)$ is not the requested output syntax, whereas $(-1)^{2/11} + (-1)^{-2/11}$ is. The latter expression uses ordinary complex radicals; it makes no assertion about expressibility using only real radicals.

1.1 The selected factor is essential

If P is reducible, solvability must be tested on the minimal polynomial f of the selected root, not on P as a whole. For example,

$$P(x) = (x^5 - x - 1)(x^2 - 2)$$

has a nonsolvable factor, but the selected number $\sqrt{2}$ certainly has a radical representation. The implementation removes repeated factors for indexing, selects a root, and only then obtains its irreducible minimal polynomial. All conjugates of one irreducible polynomial have the same radical-solvability status, although their branch multipliers differ.

1.2 Four logically different outcomes

Result	Meaning
<code>Success</code>	A radical expression has been constructed with exact algebraic identities and root selection.
<code>NotSolvableByRadicals</code>	The selected irreducible polynomial has a computed non-solvable Galois group.
<code>NotFound</code>	The finite shortcuts were explicitly requested and did not produce an answer. No impossibility claim is made.
<code>ResourceLimit</code> <code>BackendFailure</code>	or A budget was exhausted or an algebraic backend operation failed. Solvability remains unresolved.

Malformed mathematical input raises a Python `ValueError`; the command-line worker reports `InvalidInput`. The Wolfram interface uses `Failure` objects rather than leaving an expression that might be mistaken for a successful radical conversion.

2 What the supplied notebook contributes

The uploaded archive contains `FindExtension.nb`. It was inspected as inert notebook data, not evaluated. A small box-expression reader recovered 220 input cells. The inspection found several complementary exploratory ideas, not one complete decision-and-construction procedure.

`findExtension` and `findExtension3` search for lower-degree factor coefficients by taking polynomial remainders and eliminating unknowns. `project` examines real and imaginary parts of conjugates for degree reduction. `factor` and `solve` try to identify the relevant factor or conjugate. The random `fuzz` search forms sums of conjugates and uses high-precision reconstruction to suggest lower-degree quantities. Finally, `simplify` tries to improve a proposed expression with progressively more expensive simplification.

These are useful discovery methods. However, a lower-degree intermediate field need not itself provide a radical generator; a small residual numerical error does not identify a conjugate exactly; and the notebook's `While[True]` searches do not supply a stopping criterion. Some helpers also rely on private simplifier names. In particular, the inspected text contains both `Simplify[SimplifyCount]` and a spelling `Simplify[SimpifyCount]`; the latter deserves attention independently of any mathematical algorithm.

The new implementation retains the mathematical ideas of intermediate-field traces and low-degree factorization, but separates discovery from certification. Its completeness argument does not depend on random sampling, successful integer-relation reconstruction, a simplification complexity function, or an undocumented Wolfram function. It does not attempt to reproduce every notebook experiment or establish that every number appearing there is solvable. A provenance file records the original notebook's hash and the inspection method.

3 The Galois criterion and its constructive content

Let $f \in \mathbb{Q}[x]$ be the irreducible minimal polynomial of α , let $E \subset \mathbb{C}$ be its splitting field in the chosen complex embedding, and put $G = \text{Gal}(E/\mathbb{Q})$.

Theorem 3.1 (Galois criterion). *The number α belongs to a radical extension of $\mathbb{Q}$ if and only if G is solvable.*

The classical theorem, including cyclic extensions after adjoining roots of unity, is treated in [3, Proposition 5.27 and Theorem 5.34]. For a single irreducible polynomial, conjugation carries a radical expression for one root to radical expressions for all its roots. Conversely, a radical tower can be enlarged by roots of unity and conjugates to a normal tower with abelian steps; its relevant Galois quotients are solvable. The remainder of this report makes the sufficient direction computationally explicit.

A group name alone does not construct the answer. Likewise, a tower whose relative minimal polynomials have degrees $2, 3, 5, \dots$ is not automatically a radical tower: a general degree-five equation is not a fifth-root operation. The needed extra structure is a *cyclic* extension over a field already containing the appropriate roots of unity.

3.1 Choice of roots of unity

Write

$$m = \prod_{p \mid |G|} p, \quad \zeta = (-1)^{2/m}, \quad K = \mathbb{Q}(\zeta), \quad L = EK.$$

Only distinct prime factors occur in m . This is sufficient because we will refine every group quotient to prime order, and K contains $\zeta_p = \zeta^{m/p}$ for every required prime p .

Both E and K are normal over $\mathbb{Q}$, so L is normal as well. Restriction injects $H = \text{Gal}(L/K)$ into G ; consequently H is solvable whenever G is. There is no need to assume $E \cap K = \mathbb{Q}$. In fact, failing to account for that intersection is a common source of wrong degree counts. The implementation checks

$$|H| = [L : K] = \frac{[L : \mathbb{Q}]}{\varphi(m)}.$$

The rational case $|G| = 1$ is handled before this construction.

4 Concrete fields and their automorphisms

4.1 Constructing the splitting field

Start with $E_0 = \mathbb{Q}$. Factor f over E_j . If all factors are linear, stop. Otherwise adjoin a root of a nonlinear factor, preferably a root supplied by an exact quadratic, cubic, or quartic formula. When such a shortcut is not available, choose an original root of f not yet contained in E_j .

Every adjunction is a root of f , lies in the fixed splitting field, and strictly increases the current field. Once all original roots belong to the field, the algorithm stops. This is an effective finite procedure using exact factorization, field membership, and primitive-element construction. It is not a claim that these operations are cheap.

4.2 Rational power coordinates

Represent a number field as

$$F = \mathbb{Q}(\theta) \cong \mathbb{Q}[T]/(q(T)), \quad d = \deg q.$$

Its power basis $1, \theta, \dots, \theta^{d-1}$ makes every element a rational column vector. Field multiplication is polynomial multiplication followed by reduction modulo q . If $a \in F$, write M_a for the rational matrix of multiplication by a .

If $F/\mathbb{Q}$ is normal, factoring q over F gives all d images of θ . For each image b , substitution $\theta \mapsto b$ defines an automorphism. Composition and equality are computed by exact polynomial substitution in F . The result is a finite multiplication table, with the identity numbered zero. No numerical recognition of a permutation is used here.

4.3 A cheaper relative-automorphism construction

Recomputing all automorphisms by factoring the minimal polynomial of a primitive element of $L = E(\zeta)$ can be much more expensive than doing so for E . The following construction avoids that second factorization.

Primitive-element computation supplies rational c_0, c_1 such that

$$\theta = c_0 e + c_1 \zeta, \quad E = \mathbb{Q}(e), \quad L = \mathbb{Q}(\theta).$$

For every $\sigma \in G$, the only possible extension fixing ζ sends θ to $b_\sigma = c_0 \sigma(e) + c_1 \zeta$. In the rational power model of L , test the following identities:

$$\begin{aligned} q_L(b_\sigma) &= 0, \\ e(b_\sigma) &= \sigma(e), \\ \zeta(b_\sigma) &= \zeta. \end{aligned}$$

Here $e(T)$ and $\zeta(T)$ mean their coordinate polynomials in θ . All three tests take place inside one exact field.

Proposition 4.1. *The retained images are exactly the automorphisms of L fixing K .*

Proof. A root b_σ of the irreducible polynomial q_L defines a $\mathbb{Q}$ -embedding $L \rightarrow L$, hence an automorphism. The second and third identities establish the specified action on the two generators. Conversely, any automorphism fixing K restricts to an element of G , and its action on θ is forced to be the stated linear combination. Thus no required automorphism is lost. $\square$

The implementation additionally checks the expected group order $[L : \mathbb{Q}]/\varphi(m)$ and closure of the resulting table. This is an exact extension test, not an assumption of linear disjointness.

5 Prime cyclic quotients without subgroup enumeration

The derived series $G^{(0)} = G$, $G^{(j+1)} = [G^{(j)}, G^{(j)}]$ decides solvability. A nontrivial stationary term proves nonsolvability. This finite table algorithm is the general fallback; a small-degree SymPy Galois-group routine is used only as an optional earlier negative decision. Its documented degree restriction is therefore not a restriction on the general construction [5].

For a solvable subgroup J , put $D = [J, J]$. Then J/D is finite abelian. Starting with D , greedily adjoin elements while the generated subgroup remains proper in J . When no further element can be adjoined, the result J' is a maximal proper subgroup containing D . Consequently J' is normal in J , and J/J' is a simple finite abelian group, hence cyclic of prime order. Repeating gives

$$H = H_0 \triangleright H_1 \triangleright \cdots \triangleright H_s = 1, \quad H_{i-1}/H_i \cong C_{p_i}.$$

Normality is required only in the immediately preceding group. Requiring all H_i to be normal in the original G would impose an unnecessary and sometimes impossible restriction.

Set $F_i = L^{H_i}$. The corresponding ascending field tower satisfies

$$K = F_0 \subset F_1 \subset \cdots \subset F_s = L, \quad [F_i : F_{i-1}] = p_i.$$

Each extension F_i/F_{i-1} is cyclic and contains ζ_{p_i} in its base.

6 Constructing a radical generator by a nullspace

Fix adjacent groups $J = H_{i-1}$ and $N = H_i$, with $[J : N] = p$. Choose $\tau \in J \setminus N$; its restriction generates $\text{Gal}(F_i/F_{i-1})$. For an automorphism g , let A_g be its rational matrix on the power basis of L .

Consider the simultaneous rational linear system

$$(A_h - I)v = 0 \quad (h \text{ in a generating set of } N), \quad (A_\tau - M_{\zeta_p})v = 0. \quad (1)$$

The corresponding element $r \in L$ is fixed by N and satisfies $\tau(r) = \zeta_p r$.

Theorem 6.1 (The Kummer step). *The system (1) has a nonzero solution. Every such solution satisfies*

$$r^p \in F_{i-1}, \quad F_i = F_{i-1}(r).$$

Proof. Choose a normal-basis element b for the cyclic extension F_i/F_{i-1} . The sum

$$R = \sum_{j=0}^{p-1} \zeta_p^{-j} \tau^j(b)$$

is nonzero by linear independence of the normal basis and obeys $\tau(R) = \zeta_p R$. Since $R \in F_i$, it is also fixed by N , proving existence of a nonzero solution to the rational system.

For any nonzero solution r , the element r^p is fixed by N and by τ . Together these generate J , so $r^p \in F_{i-1}$. If r belonged to F_{i-1} , then $r = \tau(r) = \zeta_p r$, contradicting $r \neq 0$ and $\zeta_p \neq 1$. Since the extension has prime degree, any element outside F_{i-1} generates F_i . $\square$

The key algorithmic point is that no normal basis has to be found: ordinary rational nullspace computation constructs the required eigenvector. The multiplication matrix M_{ζ_p} is essential. Treating ζ_p as a rational scalar in a rational matrix equation would be incorrect.

6.1 Smaller generators by finite trace projections

A rational nullspace basis can have unattractive denominators. The code also tries two deterministic alternatives from powers of the original generator e of E . For $b = e$ or e^2 , when available, form

$$T_N(b) = \sum_{h \in N} h(b), \quad R(b) = \sum_{j=0}^{p-1} \zeta_p^{-j} \tau^j(T_N(b)).$$

A nonzero $R(b)$ lies in the same eigenspace. The program compares the rational coefficient sizes of r^p for these candidates and for the nullspace basis vectors, then chooses a small radicand. Explicit fixed-field and eigenvector identities are checked again for the chosen candidate.

This is a finite expression-size heuristic, not a random search and not a completeness assumption. The nullspace always supplies a candidate. In the cyclic-quintic test, this simple change replaces a very large coordinate formula by a short fifth-root expression displayed in Section 11.3.

7 From field generators to a radical expression

7.1 Maintaining an exact tower basis

Initially use the rational basis

$$\mathcal{B}_0 = (1, \zeta, \dots, \zeta^{\varphi(m)-1})$$

of K . Suppose $\mathcal{B}_{i-1}$ is already represented both as exact internal field elements and as expressions in earlier radical variables. Solve the rational coordinate system

$$r_i^{p_i} = \sum_{b \in \mathcal{B}_{i-1}} c_b b.$$

Its symbolic counterpart is the next radicand a_i . Extend the basis by

$$\mathcal{B}_i = \{br_i^j : b \in \mathcal{B}_{i-1}, 0 \leq j < p_i\}.$$

Theorem 6.1 proves linear independence and spanning. Thus the new basis size is exactly $[F_i : \mathbb{Q}] = [L : \mathbb{Q}]/|H_i|$, an invariant checked by the program.

If α is already fixed by H_{i-1} , stop before adding another radical. Otherwise continue until $H_s = 1$. A final rational coordinate solve expresses α in the accumulated basis. The implementation never hides an unresolved algebraic number in a coefficient of the final expression. Internal field generators may still occur in the audit metadata, which is not the returned radical expression.

7.2 Choosing the complex branch

Let $a = r^p \neq 0$ in the concrete embedded field. There is a unique $k \in \{0, \dots, p-1\}$ such that

$$r = \zeta_p^k a^{1/p}, \quad (2)$$

where $a^{1/p}$ is the principal complex power. After identifying k , emit that multiplier explicitly. In particular, a principal cube root of a negative real number must not silently be interpreted as the real cube root. The code does not use **PowerExpand**.

To establish the precondition for exact root matching, compute the minimal polynomial of a from the square-free part of the characteristic polynomial of M_a . Indeed,

$$\det(TI - M_a) = \text{minpoly}_a(T)^{[L:\mathbb{Q}(a)]}.$$

Both the known internal r and every candidate in (2) are roots of $\text{minpoly}_a(T^p)$. Thus branch selection is a comparison of *known roots of a known rational polynomial*, not a test that a residual is numerically small.

7.3 Separation-bound comparison

Let q be a square-free rational polynomial and let $\delta > 0$ be a certified lower bound for separation of its distinct complex roots. Assume that u, v are already known to be roots of q . Evaluate each to absolute error less than $\epsilon < \delta/8$, producing exact dyadic numbers $\tilde{u}, \tilde{v}$. Then

$$\begin{aligned} u = v &\implies |\tilde{u} - \tilde{v}| < \delta/4, \\ u \neq v &\implies |\tilde{u} - \tilde{v}| > 3\delta/4. \end{aligned}$$

Comparison of their squared distance with $\delta^2/4$ therefore decides equality. Only rational arithmetic is needed after evaluation. The code uses SymPy's exact Mignotte separation bound and bounded-error evaluation primitive, with these explicitly separated thresholds; see the underlying polynomial and root-isolation facilities in [6]. The precondition is checked by minimal-polynomial divisibility for arbitrary comparison inputs, or supplied by exact construction identities for internal calls.

This depends on the correctness of the algebraic and bounded-error backend; it is not a formally verified interval-arithmetic kernel. Ordinary decimal approximations are used elsewhere only to order candidate work. They do not certify equality. The dependency on SymPy 1.14's internal bounded-error API is explicit and motivates the pinned environment and regression tests.

8 A separate arithmetic-tower checker

The output records the prime indices, phase choices, radicands, local assignments, and final polynomial expression. Besides checking identities during construction, **verify_tower_result** independently verifies the emitted arithmetic tower without constructing any number field or using the discovered automorphisms.

Let Z represent ζ and U_i the emitted generators. The checker first checks the assignment syntax and phase formula, and then reduces the target polynomial modulo

$$\Phi_m(Z), \quad U_1^{p_1} - A_1(Z), \quad \dots, \quad U_s^{p_s} - A_s(Z, U_1, \dots, U_{s-1}). \quad (3)$$

Here each A_i is the exact earlier-basis radicand. The root-of-unity phase disappears upon taking the p_i th power. Reduction starts with U_s and proceeds backwards to Z . The relations are monic

in their newly introduced variables, so polynomial remainders suffice. Horner evaluation of f with reduction after each multiplication avoids unnecessarily large expanded powers.

If the remainder is zero, the actual emitted radical expression satisfies f because its assignments satisfy every relation in (3). The separation test then identifies it with the selected α . This checker is logically distinct from the splitting-field search. It does not prove the Galois-group computation correct and it is not a proof object checked by Lean or another formal kernel; it is an independently executable exact arithmetic check of a positive answer. On large examples, running the checker can cost more than constructing the answer.

9 The complete algorithm and its guarantee

```

Input: an exact selected algebraic root alpha
f := the irreducible minimal polynomial of alpha over Q
Try finite exact structural shortcuts unless disabled
E := splitting field of f in the selected complex embedding
G := all automorphisms of E/Q, with an exact group table
If the derived series of G is nontrivially stationary:
    return NotSolvableByRadicals
m := product of the distinct prime divisors of |G|
L := E(zeta_m)
H := automorphisms of L fixing zeta_m, by extension tests
Refine H to a series with prime cyclic quotients
For each adjacent pair, unless alpha is already in the fixed field:
    compute a nonzero Kummer eigenvector by rational linear algebra
    optionally improve its radicand with finite trace projections
    express its prime power in the earlier tower basis
    select its exact principal-power phase
    extend the tower basis and append a radical assignment
Express alpha in the final basis and return the radical circuit

```

Theorem 9.1 (Completeness of the mathematical algorithm). *Assume exact, terminating algorithms for rational and number-field factorization, primitive elements, field membership, rational linear algebra, and algebraic root isolation. With no resource bounds, the construction terminates for every exact algebraic input. It returns a radical expression for the selected number exactly when such an expression exists.*

Proof. The splitting-field stage adjoins finitely many roots of f . Automorphism computation and all subsequent group operations operate on finite sets. The derived-series criterion correctly distinguishes solvable and nonsolvable groups. In the solvable case, prime-index refinement terminates because the group order decreases at every step. The relative-automorphism construction accounts for the cyclotomic intersection exactly.

Theorem 6.1 supplies a nonzero generator at every required step. The basis invariant guarantees each radicand coordinate solve and the final target coordinate solve. A branch candidate exists and is unique because all p_i th roots of a nonzero number differ by the included roots of unity. Induction on the assignments identifies each symbolic generator with its concrete field element; the last coordinate identity therefore gives the specified α . In the nonsolvable case, the Galois criterion excludes any radical expression. $\square$

The theorem describes an effective algorithm, not an assertion that every backend call in a particular CAS version is bug-free. The implementation reports algebraic failures separately

and enables budgets by default. It contains no hard degree-six limit on the general procedure. Conversely, setting the limits to infinity does not make large instances practical.

10 Finite shortcuts that matter in practice

The complete construction is a fallback. A small exact structural identity can be much more useful than an explicit normal closure.

10.1 Centered powers and polynomial composition

After translating away the coefficient of x^{n-1} , compute the greatest common divisor g of the nonzero positive exponents. If $g > 1$, the polynomial is $h(x^g)$. Solve h recursively and take every correlated g th-root phase. For example, $(x+1)^7 + 2$ immediately gives its real root as $-1 - 2^{1/7}$. Exact polynomial decomposition is another route: if $f = f_1 \circ \dots \circ f_r$, solve successive lower-degree equations, checking the composition identity before accepting it.

10.2 Dickson polynomials

Define

$$D_0(x, a) = 2, \quad D_1(x, a) = x, \quad D_n(x, a) = xD_{n-1}(x, a) - aD_{n-2}(x, a).$$

Induction proves

$$D_n(u + a/u, a) = u^n + (a/u)^n.$$

Thus $D_n(x, a) = b$ is solved by choosing a nonzero root v of $v^2 - bv + a^n = 0$, putting $u = v^{1/n}$, and taking

$$x_k = \zeta_n^k u + \frac{a}{\zeta_n^k u}, \quad 0 \leq k < n.$$

The two summands are linked. Choosing two n th roots independently destroys the required product and can produce incorrect answers.

10.3 Reciprocal lifts

For a monic degree- $2d$ polynomial, test identities of the form

$$f(x) = x^d h(x + a/x), \quad a \in \mathbb{Q}.$$

The constant term must equal a^d . The code enumerates the rational solutions of this equation, recovers the coefficients of h by triangular coefficient elimination, and accepts only a zero residual. A root s of h then gives

$$x = \frac{s \pm \sqrt{s^2 - 4a}}{2}.$$

Failure to find such a rational a is merely failure of this shortcut.

10.4 Pair-sum resolvents

For monic f of degree n with roots $\alpha_1, \dots, \alpha_n$, put

$$S(s) = \prod_{i < j} (s - \alpha_i - \alpha_j).$$

A direct product expansion gives

$$\text{Res}_x(f(x), f(s-x)) = 2^n f(s/2) S(s)^2. \quad (4)$$

The diagonal terms give the first factor; each unordered off-diagonal pair occurs twice. The code divides out the diagonal factor and checks the square multiplicities exactly. If a low-degree factor supplies a radical pair sum s , then

$$\gcd(f(x), f(s-x)) \in \mathbb{Q}(s)[x]$$

can reveal a small factor containing the desired root. Collisions between pair sums do not invalidate (4); multiplicities must be retained. The implemented search uses input degree at most eight and resolvent factors of degree at most four. These limits affect this shortcut, not the general algorithm.

10.5 Cyclotomic polynomials and real traces

If $f = \Phi_m$, every root is explicitly ζ_m^k with $(k, m) = 1$. The code also recognizes real cyclotomic generators by testing whether $x^{\deg f} f(x + x^{-1})$ is cyclotomic; then roots are $\zeta_m^k + \zeta_m^{-k}$.

Order recognition is finite. Since

$$\frac{\varphi(m)^2}{m} = \prod_{p^a \parallel m} p^{a-2} (p-1)^2 \geq \frac{1}{2},$$

a cyclotomic polynomial of degree d has $m \leq 2d^2$. The inequality follows factor by factor: only $p = 2, a = 1$ contributes less than one, and that contribution is $1/2$. An exact cyclotomic test precedes this bounded search.

All these constructions establish that their candidates satisfy the input polynomial before root matching. Recomputing the minimal polynomial of each large nested radical is optional, not logically necessary. The low-degree base cases trust the CAS's exact degree-at-most-four formulas; higher steps check the stated polynomial identities. The regression report distinguishes these structural checks from additional minimal-polynomial checks.

11 The two motivating examples

11.1 A sextic that is only a cubic plus a quadratic

The first example is

$$f(x) = x^6 + x^4 - x^3 - x^2 - 1.$$

Its structure is visible in the exact identity

$$f(x) = x^3((x - x^{-1})^3 + 4(x - x^{-1}) - 1). \quad (5)$$

Set

$$c = \left(\frac{9 + \sqrt{849}}{18} \right)^{1/3}, \quad s = c - \frac{4}{3c}.$$

The quantity c is positive real. Since c^3 solves $v^2 - v - 64/27 = 0$, expansion gives $s^3 + 4s - 1 = 0$. That cubic has exactly one real root: its derivative is $3s^2 + 4 > 0$. Equation (5) then shows that the two real roots of f are

$$\frac{s - \sqrt{s^2 + 4}}{2} < 0, \quad \boxed{\alpha = \frac{s + \sqrt{s^2 + 4}}{2} > 0}.$$

Hence the boxed expression is the requested second real Wolfram root:

```
Root[-1 - #^2 - #^3 + #^4 + #^6 &, 2]
```

No numerical guessing is involved. The other four roots are obtained by using the other two cubic roots and both quadratic signs, followed by exact root matching.

The pair-sum route reaches the same intermediate cubic. The computed resolvent is

$$S(s) = (s^3 + 4s - 1)(s^{12} - s^9 + 8s^8 - 2s^7 - s^6 + 4s^5 + 18s^4 + 9s^3 + 2s^2 - 1).$$

For a root s of the cubic factor, the relevant common factor is $x^2 - sx - 1$. This illustrates why the notebook's intermediate-extension ideas can be effective, while the reciprocal identity is the simpler certificate.

11.2 The solvable quintic and correlated fifth roots

The second example is

$$P(x) = 5x^5 - 25x^3 + 25x + 6 = 5D_5(x, 1) + 6.$$

Take

$$u = \left(\frac{-3 + 4i}{5} \right)^{1/5}$$

with its principal value. The five roots are

$$x_k = \zeta_5^k u + (\zeta_5^k u)^{-1}.$$

The largest real root is $x_0 = u + u^{-1}$, giving

```
Root[6 + 25 # - 25 #^3 + 5 #^5 &, 5]
== ((-3 + 4 I)/5)^(1/5) + ((-3 + 4 I)/5)^(-1/5)
```

For completeness, write $(-3 + 4i)/5 = e^{i\theta}$ with $\pi/2 < \theta < \pi$. Then $u = e^{i\theta/5}$ and every x_k is real. Among the five angles modulo 2π , $\theta/5$ is closest to zero, so x_0 is largest. This angle argument explains the ordering; the software still performs exact selected-root comparison.

11.3 A genuine degree-five Galois construction

To exercise the fallback rather than a recognition formula, the tests force `method="galois"` on

$$g(x) = x^5 + x^4 - 4x^3 - 3x^2 + 3x + 1.$$

Its computed splitting-field group has order five. The cyclotomic augmentation has rational degree twenty and relative group order five. With $z = (-1)^{2/5}$, the implemented trace-guided construction for the largest real root produces

$$u = z^3(385z^3 + 110z^2 + 220z - 66)^{1/5},$$

and the following polynomial in u and z :

$$\begin{aligned} \alpha = & \frac{u^4}{6655}(26z^3 + 16z^2 + 41z + 6) \\ & + \frac{u^3}{605}(-10z^3 - 7z^2 + 2z - 4) \\ & + \frac{u^2}{55}(4z^3 + 2z^2 + 2z + 1) + \frac{uz - 1}{5}. \end{aligned}$$

The separate checker verifies $g(\alpha) = 0$ by polynomial remainders modulo $\Phi_5(z)$ and $u^5 - (385z^3 + 110z^2 + 220z - 66)$, then checks the chosen embedding. The example demonstrates an actual fifth-root Kummer step, not an invocation of a general quintic radical solver. In automatic mode the much shorter real-cyclotomic-trace shortcut is preferable.

12 Using the software

12.1 Python interface

The engine was tested under Python 3.13.5 and SymPy 1.14.0. Install the pinned dependency and run from the archive root:

```
python -m pip install -r requirements.txt
python tests/test_radical_roots.py
```

A library call returns a structured result:

```
import sys
sys.path.insert(0, "python")
import sympy as s
from radical_roots import radicalize, verify_tower_result
x = s.Symbol("x")
r = radicalize(x**6 + x**4 - x**3 - x**2 - 1, index=1)
assert r.status == "Success"
print(r.to_wolfram())
print(r.expanded())
```

The Python index is zero-based in SymPy's ordering of the distinct roots. It must not be obtained by subtracting one from an arbitrary Wolfram complex root index. The optional rational-rectangle selector avoids this convention entirely.

The command-line interface accepts a JSON array of rational coefficient strings, highest degree first, rather than executable Python expressions:

```
python python/radical_roots.py --coefficients "[1,0,1,-1,-1,0,-1]" --index 1 --timeout 300
```

Use `-method galois` to force the general construction. Defaults are a field-degree cap of 128 and a 300-second subprocess watchdog. The CLI uses `-max-degree 0 -timeout 0` to disable both; the library uses `max_degree=None`. Disabling limits can consume substantial time and memory. The degree cap is checked after constructing an extension, so it is not a strict bound on transient work or memory.

12.2 Wolfram Language interface

Keep the extracted directory structure intact. In Wolfram Language:

```
Get["/full/path/to/RadicalRoots/wolfram/RadicalRoots.wl"];
r = Root[-1 - #^2 - #^3 + #^4 + #^6 &, 2];
a = RootToRadicals[r, "PythonExecutable" -> "/full/path/to/python"];
RootReduce[a - r]
```

On Windows, supply the full path to `python.exe` when it is not on the process search path.

The metadata interface is `RadicalRepresentation[r]`. To read a saved Python result, use `ReadRadicalJSON[file,r]`; it applies the same Wolfram verification without launching Python again.

The front end proposes a dyadic rectangle using an approximation, proves that the selected input lies strictly inside it using exact algebraic comparisons, and checks its root count with `CountRoots`. The documented complex-rectangle interpretation is used [7]. Python independently rejects boundary roots and requires a unique root in the box. This avoids assuming compatible complex-root ordering across the two systems.

The returned JSON expression is an arithmetic tree with tags for rationals, i , addition, multiplication, rational powers, and earlier local variables. It is decoded without evaluating source-code strings. Process invocation uses an argument list, not a shell command. Finally, `RootReduce` checks exact equality to the original selected value [8]. A verification timeout is reported as a resource limit, not as a successful conversion. The Wolfram integration remains unexecuted in this release; the supplied `Tests.wlt` is intended for validation in a real kernel.

13 Validation and honest limitations

The reproducible Python suite reports 46 passing tests out of 46. It records the Python and SymPy versions, the source SHA-256, per-test outcomes, timings, and which answers received an additional minimal-polynomial check. Galois-Kummer results also receive the separate triangular-polynomial tower check. Eight additional command-line and certificate-rejection checks passed and have their own JSON report.

Coverage includes all six conjugates of the motivating sextic, all five roots of the motivating quintic, real and nonreal pure-cubic roots, quadratic and biquadratic extensions, a dihedral quartic, and a forced cyclic-quintic construction. It also includes polynomial composition, a shifted seventh power, cyclotomic recognition, repeated roots, selected irreducible factors, a nonsolvable S_5 example, explicit resource exhaustion, rejection of inexact or symbolic coefficients, and rational rectangle selection including boundary rejection. These are regression cases, not exhaustive certification of all number fields.

An earlier version that refactored the augmented degree-twenty field hit a 180-second watchdog on the cyclic quintic. The relative-automorphism method removed that bottleneck, and trace-guided generator selection dramatically reduced the resulting expression. These observations explain design choices; they are not a controlled benchmark against Mathematica, SageMath, Magma, or other computer algebra systems.

The dominant remaining limitation is explicit normal-closure construction. A degree- n input can have a splitting field of degree up to $n!$, and a solvable input can still produce large fields and coefficients. Rational nullspaces, automorphism tables, basis coordinates, and principal-power verification may all become expensive. Very large notebook examples were not comprehensively benchmarked, and minimal radical depth or globally shortest output is not optimized.

This should not be confused with a general complexity impossibility. Landau and Miller give polynomial-time solvability decision *and* radical construction with suitable straight-line-program encoding [4]. The present explicit-field reference implementation is not their algorithm and makes no polynomial-time claim. Its advantages are a direct proof, inspectable exact operations, useful structural shortcuts, and a clear distinction between a proved negative answer and an unfinished computation.

14 Directions for a stronger implementation

The most valuable next step is to avoid constructing unnecessarily large normal closures. Imprimitivity blocks, field intersections, and primitive solvable-group methods can support a Landau–Miller-style architecture. The arithmetic-tower interface can remain unchanged while discovery is replaced by a more scalable implementation.

Other improvements are compatible with the existing correctness argument: reduced integral bases for field arithmetic; lattice reduction and more systematic Kummer-generator optimization; lazy expression sharing; parallel independent exact candidate checks; persistent field-factorization caches; and a formally verified checker for the emitted arithmetic circuit and root-isolation evidence. An optional search suggested by numerical relations should only propose candidates, each of which must pass an exact certificate check before it affects a result.

The conceptual answer to the original question is therefore positive and algorithmic. The obstacle is not a missing criterion for radical expressibility. The practical task is to implement its constructive steps, control field and expression growth, and preserve the selected embedding. The supplied package addresses each of these explicitly, while keeping its unexecuted integration layer and finite testing coverage visible.

References

- [1] V. Reshetnikov, *Why is ToRadicals not able to handle all cases? Is there a workaround?*, Mathematica Stack Exchange, question 34011. <https://mathematica.stackexchange.com/questions/34011/>. The user-supplied `FindExtension.nb` was inspected separately.
- [2] Wolfram Research, *ToRadicals*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/ToRadicals.html>.
- [3] J. S. Milne, *Fields and Galois Theory*, version 5.10, 2022. In particular, Proposition 5.27 and Theorem 5.34. <https://www.jmilne.org/math/CourseNotes/FT.pdf>.
- [4] S. Landau and G. L. Miller, *Solvability by radicals is in polynomial time*, Journal of Computer and System Sciences **30** (1985), no. 2, 179–208. <https://www.cs.cmu.edu/~glmiller/Publications/Papers/LaMi85.pdf>.
- [5] SymPy Development Team, *Number Fields*, SymPy 1.14 documentation. <https://docs.sympy.org/latest/modules/polys/numberfields.html>.
- [6] SymPy Development Team, *Polynomial Manipulation Module Reference*, SymPy 1.14 documentation, including exact root comparison and root isolation. <https://docs.sympy.org/latest/modules/polys/reference.html>. The implementation also uses the 1.14 bounded-error evaluation primitive.
- [7] Wolfram Research, *CountRoots*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/CountRoots.html>.
- [8] Wolfram Research, *RootReduce*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/RootReduce.html>.

Web documentation was consulted on 8 September 2026. The software pins the actually tested SymPy version; references to current documentation do not assert compatibility with every future release.